# Detecting and Localizing Network Faults in RoCE-Based AI Clouds under Restricted Telemetry*

Sinuo Cao[1], Weiqi Zhu[1], Jie Gui[1], Jintao He[1], Qun Huang[1†], Hao Liang[2], Yang Wang[2], Bolong Cui[2], Tingfeng Jiang[2], Weiling Gu[2], Tao Peng[2], Xudong Liu[2], Ronghua Sun[2], Zhi Zhang[2], Xiangyang Wu[2], Wei Song[2]

[1]National Key Laboratory for Multimedia Information Processing, School of Computer Science, Peking University
[2]Huawei Technologies Co. Ltd

## Abstract

This paper addresses network fault detection and localization in Huawei's large-scale RoCE-based AI public cloud infrastructures. Diagnosing network faults is difficult in this scenario. Public-cloud privacy and commodity-switch hardware restrict the available diagnostic signals to network-side, standard-switch telemetry. Moreover, RoCE semantics and the throughput-sensitive training workloads we host make TCP-targeted and latency-based techniques a poor fit. To address these challenges, we present **FACT**, a production system that exploits protocol-native signals already present in the network using only commodity-switch capabilities. These signals include NAK packets for packet loss, packet conservation counters, and per-flow throughput computed from mirrored packet headers. FACT organizes these signals into a two-tier architecture that separates objective fault detection from evidence-based root-cause analysis. We have deployed FACT in multiple production infrastructures, and the longest deployment has lasted 18 months. Across all deployed infrastructures in 2025, FACT detected thousands of network faults and localized 99.95% of network faults within 10 minutes.

***CCS Concepts:*** • **Networks** → **Network monitoring**.

***Keywords:*** Network Fault Localization, Network Monitoring, RDMA, RoCE, Data Center Networks, AI Infrastructure

## 1 Introduction

This paper focuses on network fault detection and localization in Huawei's large-scale RoCE-based AI public cloud infrastructures. The scales of these infrastructures range from approximately 7,000 to over 100,000 NPUs. Network faults, manifesting as *packet loss* or *congestion*, are inevitable at such a large scale. To maintain service reliability and enhance user experience, we must achieve: (1) *accurate detection* of all fault types, including unknown-cause faults, with minimal false positives; (2) *actionable localization* that provides root-cause information for known faults and device-level localization for challenging cases; and (3) *rapid response* to network faults by localizing them quickly after occurrence.

Achieving these goals is challenging in our scenario. First, two constraints restrict the available diagnostic signals: (1) as a public cloud provider, our primary leasing model is bare-metal rental. Tenant machines are inaccessible for privacy reasons, so per-NIC counters, kernel logs, application traces, and agents deployed on hosts cannot be used for diagnosis; (2) the vast majority of our switches are standard merchant silicon devices because deploying programmable switches or custom ASICs at the scale of tens of thousands of devices is cost-prohibitive. The restrictions eliminate most diagnostic techniques the research community has developed, including host-based monitoring systems [2, 13, 16], programmable-switch telemetry [23, 36, 41], and application-level diagnosis frameworks [9, 39]. Second, two characteristics of our scenario further compound the difficulty: (1) we use RoCEv2 with PFC for lossless transport. RoCE semantics differ fundamentally from TCP, so TCP-targeted techniques cannot be directly ported; (2) the dominant workload is large-model training, whose long-lived collective flows are primarily throughput-sensitive: sustained throughput degradation extends collective completion time, while transient queueing is tolerated. Yet general-purpose congestion detection relies on latency [16, 24] or queuing metrics [40], which raise alerts for harmless transient queueing while possibly missing the sustained throughput degradation that actually slows training.

To address the challenges, we present **FACT** (*Fine-grained AI Cloud Telemetry*), a fault detection and localization system designed for RoCE-based AI public cloud infrastructures under public-cloud and commodity-hardware constraints. FACT's design is based on an observation: the network already generates the signals needed for fault diagnosis. First, NAK (Negative Acknowledgment) packets sent by RoCE receivers carry both the loss indication and the endpoint IPs needed for path-level loss localization. Second, the packet conservation property of lossless fabrics guarantees that, in the absence of faults, the number of RoCE packets entering a switch equals the number exiting it, and a mismatch directly signals an internal drop. We exploit this to obtain LossAtSwitch, a switch-level loss signal. Moreover, the `First` and `Last` packets of every RoCE flow carry the

*This is an extended version of the paper accepted for publication at *The 2026 ACM SIGOPS Annual Technical Conference (SIGOPS ATC 2026)*, https://doi.org/10.1145/3828523.3857027. This version additionally includes Appendix A. Licensed under CC BY 4.0.

†Qun Huang is the corresponding author.

metadata needed to compute flow effective throughput (FET), a throughput-based congestion signal. None of these signals requires host instrumentation or programmable hardware. Collecting them requires only standard ACLs, packet mirroring, and counters present on commodity switches.

FACT uses a two-tier architecture. Tier 1 uses fault signals (NAK, LossAtSwitch, FET) for primary detection and coarse localization, and Tier 2 draws on continuously recorded evidence (FlowPath records, device logs, performance metrics) for root-cause analysis. This two-tier structure follows from operational experience: we found that logs and performance metrics, while covering over 99% of encountered faults, fail entirely for silent faults and introduce noises, so they are better suited for root-cause labeling after a fault has been detected by other objective and precise signals. We also found that congestion diagnosis is difficult post-hoc: tenants cannot reproduce their workloads, and congestion events rarely leave log traces. Therefore, real-time recording of flow paths, throughput, and device metrics is essential.

We have deployed FACT in multiple production RoCE-based AI public cloud infrastructures. The longest deployment has lasted 18 months. On a representative infrastructure with ~12000 NPUs over one year, FACT achieved 100% detection recall: every packet loss was localized to the faulty switch, and every network-originated congestion event was traced to its root cause. Detection precision reached 99%, and 99% of faults were pinpointed to root cause. Across all deployed infrastructures in 2025, FACT localized 99.95% of network faults within 10 minutes.

Our main contributions are:

- The design and production deployment of FACT, a fault detection and localization system that operates under public-cloud and commodity-hardware constraints, covering both packet loss and congestion.
- A set of protocol-native fault signals that require only commodity-switch capabilities: NAK-based path-level loss detection, counter-based switch-level loss detection, and FET-based throughput-sensitive congestion detection.
- Insights from our experience in building a production-level fault detection and localization system. These insights provide reference for future research and practice.

## 2 Background and Motivation

In this section, we first characterize our operational scenario (§2.1) and introduce the network faults we encounter and the operational goals we should achieve in fault detection and localization (§2.2). We then derive two design challenges that any fault detection and localization system must address in our setting (§2.3). Finally, we survey related work and discuss their applicability to our scenario (§2.4).

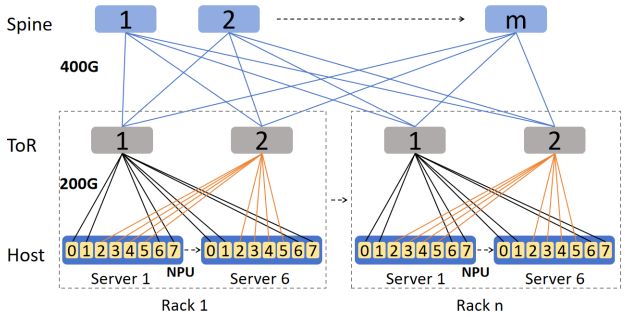



**Figure 1. Representative network topology.**

### 2.1 Scenario

Huawei Cloud operates multiple RoCE-based AI cloud infrastructures, whose scales range from approximately 7,000 to more than 100,000 NPUs. Figure 1 depicts the network topology of a representative infrastructure. The network employs a two-tier Clos [1, 7] topology. Each rack contains two top-of-rack (ToR) switches and six servers. Each server houses eight NPUs, each equipped with a dedicated RNIC. The RNIC-to-ToR links operate at 200 Gbps, and ToR-to-spine links operate at 400 Gbps. Infrastructure scale can be controlled by adjusting the number of spine switches and racks.

These infrastructures share several characteristics that define the constraints and assumptions of our scenario:

- **Public-cloud constraints.** Our primary leasing model is bare-metal rental, with entire eight-NPU servers as the basic unit leased to tenants. The scale of tenant domains ranges from tens to thousands of NPUs. For privacy reasons, we cannot access anything on tenant machines.
- **Commodity switches.** The majority of our switches are standard merchant silicon devices, while only a subset feature programmable pipelines or custom hardware capabilities. Consequently, our fault detection design must assume that switches support only universally available capabilities: access control lists (ACLs), remote packet mirroring, basic counters, and standard logging.
- **Lossless RoCEv2 with reliable transport.** We employ RoCEv2 with Priority Flow Control (PFC) [20] to implement lossless networks. Because AI workloads demand reliability, service flows operate in Reliable Connected (RC) mode with NAK (Negative Acknowledgment)-based retransmission mechanisms. The RoCE fabric is a dedicated scale-out backend. RoCE traffic does not share switches or buffers with TCP, control, or storage traffic.
- **Training-dominant workloads.** Our infrastructures primarily host large-scale AI model training workloads. These workloads generate numerous long-lived, large RoCE flows that normally drive links above 90% utilization, with DCQCN [43] as the congestion control. Therefore, the performance of training workloads is highly sensitive to network throughput.
- **Static per-flow load balancing.** Our network is non-

oversubscribed. Therefore, we use static per-flow load balancing based on ECMP [18, 19]. Specifically, we pre-configure the UDP source ports used by each NPU's RNIC before handing over the machines to tenants, and switches hash only on the UDP source port to distribute flows across ECMP paths. For example, in a ~7,000-NPU cluster with 12 spine switches, each rack connects 24 NPUs to ToR 1 and 24 NPUs to ToR 2. We configure source-port hashing such that the 24 NPUs under each ToR are grouped into pairs, with each pair mapped to a distinct ToR-to-spine 400 Gbps uplink. This yields a 1:1 convergence ratio, ensuring that under normal conditions no congestion occurs on ToR-to-spine uplinks.

- **No multicast.** In our infrastructures, there is no multicast of RoCE traffic on switches.

We argue that these characteristics do not represent a narrow scenario. Much of our hardware was originally provisioned for general-purpose RoCEv2 workloads (e.g., cloud storage) and has been repurposed to serve AI cloud demands as they emerged. We believe this situation is far from unique. Notably, not all of these assumptions are limitations of our work. For example, FACT does not require static load balancing itself, but only per-flow path affinity and a way to reconstruct each flow's path. We discuss in §10 how our approach can be adapted to dynamic per-flow load-balancing and other workload types.

### 2.2 Network faults and operational goals

We classify the network faults we encounter into two categories: packet loss and congestion.

**Packet loss.** We treat any loss in a PFC-protected RoCE network as a fault and assume no "normal" nonzero loss rate. Some losses stem from known causes such as link failures. Some are more challenging: *silent drops*, which leave no Syslog [14] or SNMP [11] trace, and *silent corruption*, where switches corrupt packet contents that pass all hardware checks and reach the receiver undetected until the RNIC discards them (similar to what is reported in [17]). The root causes of these challenging loss types remain elusive.

**Congestion.** Even in the absence of packet loss, network conditions can reduce throughput in service flows. Two primary causes dominate. *Incast* occurs when concurrent flows contend on shared links, typically due to ECMP misconfiguration or suboptimal affinity scheduling. *Receiver-side throttling* occurs when receiver endpoints fail to process packets at line rate due to overload or partial hardware degradation.

Figure 2 outlines the general workflow for handling network faults. To localize faults quickly and enable fast recovery, our fault detection and localization system should achieve the following operational goals:

1. **Accurate detection.** The system must detect all network faults, including unknown-cause faults. Equally important, it must minimize false positives to avoid alert fatigue.
2. **Actionable localization.** The system should provide information about the causes of faults. For known fault types, it should identify the root causes directly so that operators can adopt corresponding resolution methods. For challenging cases where root causes cannot be determined, the system should at least localize faults to the switch or link, enabling operators to isolate the faulty component as a temporary resolution measure.
3. **Rapid response.** The system should localize faults quickly. We set an internal target of localizing faults within ten minutes: operational experience showed that ten-minute localization keeps tenant impact acceptable. In practice most faults are localized much faster (§8.3).

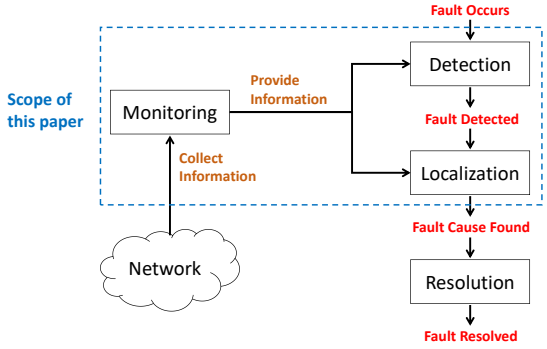



**Figure 2. General network fault handling workflow.**

### 2.3 Design challenges

The scenario (§2.1) and the goals in §2.2 bring two challenges to any network fault detection and localization system.

**Challenge 1 (C1): Operate using only network-side, commodity-switch telemetry.** The constraints in our scenario restrict the available diagnostic signals. First, due to public-cloud privacy concerns, the system must collect all diagnostic information from the network, without access to end-host counters, kernel logs, or application traces. Second, it must use only the standard capabilities on commodity switches—packet mirroring, ACLs, and basic counters.

**Challenge 2 (C2): Adapt to RoCE protocol and training workloads.** Two characteristics of our scenario bring distinct challenges. First, RoCE differs from TCP in both packet semantics and loss patterns. Diagnosis techniques built around TCP-specific fields do not understand RoCE packets, whose loss and retransmission signals are instead carried in InfiniBand headers (e.g., NAKs). Moreover, PFC changes the loss pattern itself, as packet loss no longer accompanies congestion but indicates a fault. Second, training workloads are primarily throughput-sensitive: a 10% throughput drop extends collective communication time, whereas millisecond-level queueing is tolerated. General-purpose congestion detection relies on latency [16, 24, 27] or queuing metrics [40]; both raise alerts for transient queueing that training tolerates, yet may miss the sustained throughput degradation that actually slows training. The system must therefore detect faults using indicators native to RoCE

semantics and sensitive to sustained throughput degradation.

### 2.4 Related work

We discuss the applicability of existing network fault diagnosis approaches in our setting.

**Host-side techniques.** Various work operates on end hosts, whether by sending active probes into the network [8, 16, 17, 24–26, 30, 33, 42], passively monitoring network information [2, 13, 28, 31, 32], or measuring application-level metrics [9, 15, 39]. These approaches violate (C1): they need host access we do not have. Moreover, probing-based methods detect congestion by measuring latency, which may miss the sustained throughput degradation that training suffers (C2). One recent work, ByteTracker [17], proposes agentless probing that requires only a reserved listening port on the host; we acknowledge its value but note that it targets TCP networks and has unknown RoCE applicability.

**Programmable-switch techniques.** These systems [23, 36, 37, 41] leverage programmable data planes to achieve fine-grained diagnosis. For example, LossRadar [23] exploits flow conservation by deploying ingress/egress meters based on Bloom filters to diagnose packet loss; NetSeer [41] records specific flow events in the data plane to provide evidence for root-cause analysis; HawKeye [36] traces PFC provenance to construct evidence chains for congestion. These systems offer impressive localization precision, but they require programmable hardware and cannot address (C1).

**General switch telemetry.** General switch telemetry systems [4, 22, 29, 44] can address (C1) by using only standard capabilities, but cannot be adopted as-is. For example, FlowRadar [22] performs per-flow loss detection by comparing packet counts on upstream and downstream switches. However, its flow identification relies on flowlet boundaries [34] that are nearly absent in RDMA networks [3], violating (C2). EverFlow [44] demonstrates that match-and-mirror is a powerful primitive, but choosing what to match and balancing diagnostic coverage with mirroring overhead require deployment-specific design. These systems are our closest starting point, but each needs adaptation to our scenario.

**Log and alert analysis.** These systems focus on analyzing causes of faults based on logs and alerts from various fault detection tools. Representative work includes SkyNet [38], which aggregates alerts from multiple detection tools and reasons over them to identify root causes, and BiAN [35], which applies large language models to analyze logs. These systems operate within our constraints and can provide valuable assistance for diagnosis. However, they do not solve the signal acquisition problem (C1) and cannot compensate for the lack of RoCE-aware, throughput-sensitive indicators (C2). Our own pre-FACT detection was such a system: switch logs and performance metrics matched against an operational rule list. However, silent faults left no evidence for it to match; one silent-corruption event cost over 30 hours of manual localization.

**Conceptual frameworks.** The Knowledge Plane [5] envisioned networks that discover their own faults and explain their causes under incomplete observations. FACT shares that goal, but is a scoped production system built on protocol-native signals and rules, not a cognitive architecture.

No single existing system addresses both challenges while covering both packet loss and congestion. Our goal is to build a production fault detection and localization system that uses only available capabilities, is adapted to our cluster characteristics, and achieves the operational goals.

## 3 FACT Overview

FACT is a production fault detection and localization system that satisfies the requirements of §2.3 under the scenario of §2.1. Its design stems from the insights we gained from our practical experience. We first describe these insights (§3.1), then present the overall architecture of FACT (§3.2).

### 3.1 Key insights

**Insight 1: Fault detection and root-cause analysis serve different purposes.** As is common practice, we collect logs via Syslog [14] and performance metrics via SNMP [11] or similar tools across switches. We have also accumulated a rule list that matches abnormal logs or metric anomalies with known fault causes. Over 99% of the packet losses encountered in production trigger at least one rule in this list.

However, we find that building a rule-based diagnosis system directly on top of logs and performance metrics is insufficient. First, silent faults (e.g. silent drops or corruptions) produce no observable log or metric anomalies. A rule-based system fails entirely for these cases, leaving operators to spend hours on manual diagnosis. Second, even for faults that produce logs, the alert volume is noisy: a transient event such as a brief link flap may produce a log even when no loss occurs, leaving the operator uncertain whether it reflects a genuine fault. Third, continuously scanning the full log stream imposes significant CPU overhead.

This insight led us to separate detection from root-cause analysis: detection requires objective, precise signals that fire for all faults regardless of whether they produce logs. Logs and performance metrics excel at root-cause labeling after a fault has been detected and localized to a device, but cannot serve as the primary detection signal. This separation assigns primary roles rather than forbidding overlap. Precise metrics such as PFC pause counters also assist detection.

**Insight 2: Congestion causality can only be established with real-time information.** We find that diagnosing congestion post-hoc is difficult. Most congestion events result from workload dynamics (e.g., *incast*) or end-host faults (e.g., *receiver-side throttling*), so they rarely leave explicit logs. Furthermore, congestion is tied to the specific flows active at

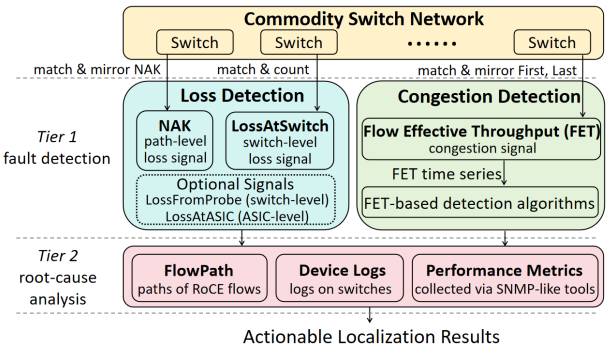



**Figure 3. Architecture of FACT. The fault detection tier (top) comprises two parallel modules operating on different fault signals. The root-cause analysis tier (bottom) draws on continuously recorded evidence. Optional signals are shown with dashed borders.**

that moment, and we cannot expect tenants to reproduce their workloads to aid debugging.

Therefore, the system must continuously record, in real time, the information needed to reconstruct the causal chain of a congestion event. Specifically, it must track which flows traverse each link, what throughput each flow achieves, when each flow is active, and other related switch metrics. When congestion is detected, the system can immediately retrieve this evidence to identify the root cause.

**Our design.** The insights motivate a 2-tier design. Tier 1 addresses Insight 1: a set of *fault signals* handles fault detection. These signals are objective, network-visible indicators that reliably flag the presence and coarse location of faults. Tier 2 addresses Insight 2: after faults are detected and coarsely localized by Tier 1, real-time evidence (logs, performance metrics, flow-path records) handles root-cause analysis. The two tiers are complementary: Tier 1 provides accurate detection, and Tier 2 achieves actionable localization.

### 3.2 FACT architecture

Figure 3 presents FACT's architecture. The detection tier (Tier 1) consists of two parallel modules: a loss detection module and a congestion detection module, each operating on different fault signals collected from switches. When Tier 1 detects a fault and localizes it to a specific device, link or path, the root-cause analysis tier (Tier 2) retrieves the relevant evidence collected from switch to pinpoint the fault cause. The output of the system is a fault alert annotated with a chain of supporting evidence, enabling operators to act within minutes. All signals shown in the figure are obtained without host access and require only commodity-switch capabilities, addressing (C1).

**Fault detection (Tier 1).** Following Insight 1, FACT selects fault signals that are objective and address (C1) and (C2). This tier directly addresses the goal of accurate detection.

- For packet loss (§4), we use two fault signals. (1) *NAK* packets are sent by RoCE receivers whenever a sequence-number gap is detected. Under our per-flow ECMP setting, a gap reliably indicates a packet loss along the specific path the flow traverses. Switches match NAK packets using ACL rules and mirror them to a central collector. (2) *LossAtSwitch* exploits the packet conservation property of lossless RoCE: every packet entering a switch must exit through a port or be internally dropped. By comparing ingress and egress RoCE packet counts at each switch, we detect any internal packet loss at the switch level, including silent drops. Together, NAK provides path-level loss indicators, while LossAtSwitch pinpoints the specific switch where loss occurred.
- For congestion (§5), FACT detects throughput degradation rather than latency increase. The key signal is *FET* (Flow Effective Throughput), defined as flow size divided by flow completion time. We compute FET by matching and mirroring only the `First` and `Last` packets of RoCE flows on switches. A set of detection algorithms monitors the FET time series of all flows and identifies paths where throughput has dropped below expected levels.

**Root-cause analysis (Tier 2) (§6).** Once a fault is detected and coarsely localized by Tier 1, FACT draws on three continuously recorded sources of evidence to determine the root cause. (1) *FlowPath* records the path (sequence of switches) traversed by every RoCE flow, acquired via packet sampling on switches and ECMP hash computation. (2) *Device logs* are collected via Syslog [14] from the switches and matched against our operational rule list for known-cause labeling. (3) *Performance metrics* are polled via SNMP [11] or similar tools from the switches. Representative metrics include PFC pause frames, interface error rates, and buffer occupancy. These three sources, combined with the fault signals from Tier 1, form a complete evidence chain that supports the goal of actionable localization.

**Optional signals.** When available, FACT can incorporate two optional signals. For tenants who grant authorization, we deploy probing agents within tenant domains to provide switch-level loss detection through active probing (*LossFromProbe*) as cross-validation. For a subset of switches with our custom ASICs, we leverage custom ASIC telemetry that reports specific silent fault types directly (*LossAtASIC*). Details are in §4.3 and Appendix A. They augment FACT's coverage and robustness but are not required for operation.

## 4 Loss Detection

This section describes the fault signals used by the loss detection module of Tier 1: NAK (§4.1) and LossAtSwitch (§4.2), followed by two optional signals available when the corresponding capabilities are present (§4.3).

### 4.1 NAK: path-level loss signal

NAK packets serve as a path-level loss indicator. In RoCE's RC mode, a receiver sends a NAK when it detects a sequence-

number gap. Under per-flow ECMP setting, all packets of a flow traverse the same path, so a sequence-number gap at the receiver reliably indicates a packet loss on that path.

We collect NAK packets by matching and mirroring them on switches. On each switch, we configure ACL rules that match NAK packets by the `OpCode` field in the BTH header and the `Syndrome` field of the AETH header. Matched packets are mirrored via ERSPAN [12] with GRE [10] encapsulation to a dedicated processing cluster.

From a mirrored NAK packet, we extract the source and destination IP addresses. The source IP is the receiver of the original RoCE flow, while the destination IP is the sender. Combined with FlowPath information in Tier 2, the two IPs yield the path of the flow that experienced loss.

**NAK reliability.** In our practical experience, we have not observed a packet loss event that failed to generate NAKs. We did encounter a few false NAKs—sporadic NAK packets occurring without genuine loss—caused by PCIe retransmissions at the NIC, intra-NIC reordering, or CPU interrupt preemption at hosts. These were present only during the initial deployment phase and then stabilized to fewer than ten occurrences per month. False NAKs are easy to separate in practice: they are sparse and trigger no other fault signal, whereas genuine faults tend to produce clustered NAKs. This distinction is empirical, so FACT never blames a device on a single NAK, and filters out sparse NAKs that exist without other fault signals.

### 4.2 LossAtSwitch: switch-level loss signal

LossAtSwitch identifies the specific switch where packet loss occurred. It leverages the *packet conservation property*: in the absence of faults, the number of RoCE packets entering a switch must equal the number exiting it. This property does not hold in traditional TCP networks, where congestion-induced packet loss and switch multicast are common. However, it holds in our setting because (1) RoCE employs PFC to theoretically eliminate congestion-induced packet loss, and (2) we do not enable multicast for RoCE traffic on switches[1].

**Challenge: in-flight packets.** Exploiting packet conservation is not as simple as comparing simultaneous ingress and egress counters. At any moment, some packets are in flight between a switch's ingress and egress pipeline. A naive comparison counts these packets at ingress but not at egress.

**Solution: dual-value marking.** We deploy an independent set of counters on each physical line-card slot of the switch, and on each slot we run a dual-value marking scheme to isolate in-flight packets from the comparison. At each slot, counter 1 and counter 2 are maintained at both ingress and egress—four counters total per slot. The two metadata values (value 1 and value 2), carried in a standard in-switch metadata field (e.g., `qos-local-id`, available on commodity switches), determine which counter is incremented.

[1]The conservation property still applies to multicast-enabled switches if multicast event counters are accessible and are taken into account.

**Table 1.** Example of packet counts across eight physical line-card slots of a spine switch during one collection interval.

| Slot | Egress Counter 1 | Ingress Counter 1 |
|---|---|---|
| 1 | 32,677,795 | 27,915,142 |
| 2 | 124,765,140 | 51,388,095 |
| 3 | 67,936,829 | 110,881,401 |
| 4 | 44,046,919 | 35,682,034 |
| 5 | 54,052,866 | 40,924,911 |
| 6 | 74,748,941 | 33,269,097 |
| 7 | 53,256,642 | 108,773,814 |
| 8 | 80,590,500 | 123,241,138 |
| **Sum** | **532,075,632** | **532,075,632** |

The operation alternates between the two values in successive intervals as follows:

1. **Mark with value 1.** The control plane configures the ingress ACL on all slots (e.g., match UDP destination port 4791 for RoCE) to set the metadata field to value 1. Counter 1 at both ingress and egress records packets carrying this mark.
2. **Switch to value 2 marking.** After a configurable collection interval (e.g., 60 seconds), the control plane changes the ingress rule to mark packets with value 2. New packets increment counter 2 at both ingress and egress.
3. **Grace period.** A short pause (e.g, 5 seconds) lets in-flight value-1-marked packets reach egress and be counted.
4. **Collect and compare.** The control plane collects the counter 1 values from all slots and separately sums the ingress and egress counts. If the two sums differ, one or more RoCE packets were dropped internally on the switch during the preceding interval.
5. **Reset and repeat.** Counter 1 is zeroed across all slots. The next interval uses value 2 as the active mark, and the cycle continues with the two values alternating.

This scheme ensures that every comparison operates on a closed set of packets—those marked with the same value and fully flushed through the pipeline—so in-flight packets never span a comparison boundary.

**Example.** Table 1 shows an example of counter values collected across the eight physical line-card slots of a production spine switch during one collection interval, with each slot housing eight ports. The total ingress and egress counts are equal, confirming that no RoCE packet is lost on this device.

**Coverage.** LossAtSwitch covers nearly all in-switch packet loss, including silent drops. Remaining gaps primarily include physical-layer errors outside the switching logic (e.g., link-layer hardware failures) and silent corruptions, which corrupt payloads rather than dropping packets.

### 4.3 Optional loss signals

When the corresponding capabilities are available, FACT can incorporate two optional loss signals that augment detection coverage and robustness but violate (C1). We describe them

for completeness.

**LossFromProbe.** For tenants who grant authorization, we deploy active probing agents within the tenant domain. We generate probe tasks that cover all network links reachable by the tenant's flows, and use network tomography to infer the most likely faulty device from path-level probe loss. Details are in Appendix A.

**LossAtASIC.** Silent packet drops typically occur within the switch data plane. On a subset of switches of our own design, we embed chip-level monitoring that detects known silent-drop patterns (e.g., missing or erroneous forwarding table entries) and reports them directly with error cause. These reports cover approximately 0.1% of all loss events.

## 5 Congestion Detection

The congestion detection module of Tier 1 relies on a single fault signal: FET (Flow Effective Throughput). We first describe how FET is computed (§5.1), then present the set of detection algorithms that operate on FET time series (§5.2), and finally discuss a practical noise-reduction step we introduced after observing false positives in production (§5.3).

### 5.1 FET computation

FET is defined as flow size divided by flow completion time (FCT). To compute FET without host instrumentation, we match and mirror only the `First` and `Last` packets of each RoCE flow from commodity switches.

The matching and mirroring mechanism is similar to that of NAK (§4.1). RoCE traffic in our infrastructures is dominated by `Write` and `Write-with-Imm` verbs. We configure ACL rules on switches to match the `First` and `Last` packets of these verbs using the BTH OpCode field. Matched packets are mirrored to a processing cluster and timestamped. For privacy compliance, no payload data is inspected.

Given the mirrored `First` and `Last` packets of a flow, FET is computed as:

$$FET = \frac{flow_size}{FCT}$$

Here, $flow_size = packet_count \times MTU$, where $packet_count$ is derived from the difference between the PSN (Packet Sequence Number) fields in the last and first packets, and MTU is a pre-configured constant representing the maximum transmission unit size in bytes. *FCT* is the difference between the timestamps of the `Last` and `First` packets.

### 5.2 FET-based congestion detection

Under our static per-flow ECMP setting, the network path between a source-destination IP pair is fixed, so congestion detection reduces to identifying IP pairs that exhibit throughput degradation. The congestion detection module is activated periodically. In each detection cycle, it identifies source-destination IP pairs with new FET records, aggregates the FET values between each pair over a recent time window into a time series, and applies detection algorithms on it.

**Throughput degradation patterns.** We observe three distinct patterns of throughput degradation caused by network faults in our environment:

1. **Absolute degradation:** FET drops abruptly and consistently below a clear threshold, typically when a link fault reduces available bandwidth.
2. **Fluctuation:** FET oscillates within a range, making it difficult to determine whether a genuine fault has occurred.
3. **Relative degradation:** For flows that inherently cannot achieve line rate due to application-level limitations, throughput drops relative to their own historical baseline rather than crossing a fixed threshold.

We design three detection algorithms, each targeting one of these patterns. An IP pair is reported as congested if detected by any of the three.

**Algorithm 1: Sliding window sampling.** This algorithm targets absolute degradation (Figure 4(a)). It counts the proportion of FET samples below a predefined threshold (e.g., 0.9× line rate) within a sliding window (e.g., the latest 60 samples). If the proportion exceeds a specified ratio (e.g., 0.8), the IP pair is reported as congested.

This algorithm is lightweight and works with short FET sequences (a few minutes), but cannot reliably handle fluctuations near the threshold and may produce false positives from transient dips.

**Algorithm 2: STL decomposition.** This algorithm targets fluctuation patterns (Figure 4(b)). It applies STL (Seasonal and Trend decomposition using Loess) [6] to the FET time series, isolating the trend component that captures long-term throughput changes while filtering out seasonality and short-term fluctuations. The algorithm computes the average FET over each detection cycle (e.g., one minute) and applies STL over a longer window (e.g., one hour). If the proportion of trend values below a threshold exceeds a specified ratio, the IP pair is reported as congested.

This algorithm achieves high accuracy even under significant FET fluctuation but requires longer sequences (typically 30 minutes or more) and incurs higher computational overhead due to the complexity of the decomposition.

**Algorithm 3: Dynamic baseline.** This algorithm targets relative degradation (Figure 4(c)). It maintains an adaptive baseline for each source-destination IP pair computed from historical FET data. If recent FET values consistently fall below this baseline, the IP pair is reported. Detection proceeds in two recurring phases:

*(1) Baseline update.* For each IP pair, the algorithm maintains a sliding window of recent per-cycle average FET values, $S = \{x_1, \ldots, x_n\}$. A dynamic lower bound $L_t$ is computed as follow:

$$L_t = \min\big(\ \mathrm{med}(S) - \beta \cdot (\mathrm{med}(S) - Q_{0.2}(S)),\ \gamma \cdot Q_{0.2}(S)\big)$$

where $\mathrm{med}(S)$ is the median, $Q_{0.2}(S)$ is the 20th percentile of $S$, $\beta$ (e.g., 1.5) controls sensitivity to dispersion in the lower half of the data, and $\gamma$ (e.g., 0.9) provides a floor relative to the

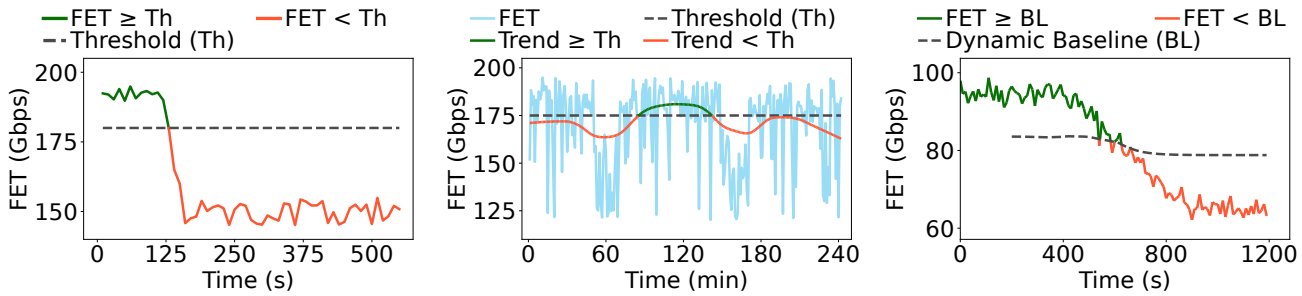



**Figure 4. Illustration of the three congestion detection algorithms: (a) Sliding window sampling for absolute degradation, (b) STL decomposition for fluctuation handling, (c) Dynamic baseline for relative degradation.**

lower percentile. To avoid abrupt changes, $L_t$ is smoothed over time via exponential moving average.

*(2) Anomaly decision.* A point anomaly is flagged in cycle $k$ if $FET_k < L_t$. A congested-path alert is raised when the number of point anomalies within a lookback window of $T$ cycles exceeds $\tau$, where $T$ (e.g., 100) and $\tau$ (e.g., 60) are configurable thresholds.

This algorithm effectively detects degradation for flows that cannot reach line rate, but it requires an initial learning period to establish a reliable baseline and may miss very short-lived anomalies due to the persistence requirement.

### 5.3 Noise reduction in production

While the three algorithms performed well in our testbed, initial production deployment revealed a non-negligible false-positive rate (details in §8). Investigation traced the root cause to timestamp inaccuracy: the ERSPAN [12] we use does not support timestamping on the switch itself, so mirrored packets are timestamped at the processing cluster. Network jitter on the mirror path introduces noise into the computed FCT, which distorts FET values and triggers false positives.

To address this, we added a severity filter before starting the Tier 2 root-cause analysis. Among the IP pairs reported as congested, only those meeting at least one of the following criteria are considered severe and forwarded:

1. PFC pause frames are observed on the affected path during the congestion period, indicating that backpressure has happened. The affected path is obtained from FlowPath and PFC counters are from performance metrics, both are continuously recorded for Tier 2 and reused here at zero additional overhead.
2. The tenant reports application-level performance degradation and the reported IP pair falls in that tenant domain.

This filter reduced the false-positive rate to below 10% in production. Notably, for sustained congestion, the false-negative rate is near zero because periodic detection makes the per-interval miss probability decay exponentially.

We consider this trade-off acceptable. Transient congestion is a natural consequence of bursty AI workloads and is tolerated without performance impact. Only sustained congestion degrades training linearity, which is the metric we care about. Filtering transient events therefore eliminates noise without sacrificing operational value.

## 6 Root-Cause Analysis

This section describes Tier 2 of FACT: how fault signals from Tier 1 are combined with continuously recorded evidence to determine root causes. We first describe the three evidence sources—FlowPath, device logs, and performance metrics (§6.1, §6.2)—then present the root-cause analysis workflows for loss (§6.3) and congestion (§6.4).

### 6.1 FlowPath: recording paths of RoCE flows

FlowPath records the sequence of switches traversed by each RoCE flow. We use a lightweight approach based on packet sampling. Switches sample RoCE packets at a low rate (typically 1 in 10,000) and mirror the samples to a processing cluster. Sampled packets are aggregated by 5-tuple and switch ID to reconstruct the full path of each flow. Because AI training flows are large, the sampling method captures complete paths for the vast majority of flows (over 95%). For the remaining cases where a path is incomplete due to missed samples, we infer the missing hops by computing the paths selected by the configured ECMP hash function. Note that sampling applies only to path reconstruction. The sampling rate thus affects only how paths are reconstructed (i.e., how often the hash-based inference is needed), not detection latency, as the detection signals in Tier 1 are collected without sampling.

We initially attempted pure hash-based path computation, but a firmware issue on switches made it unreliable at that time. We therefore adopted sampling as the primary method and retained it even after the hash issue was resolved, as it provides an independent source of truth.

FlowPath records carry timestamps and support two query modes: (1) querying the path of a specific RoCE flow; and (2) querying the path between a source-destination IP pair during a specified time period, if at least one flow between them was active.

**Algorithm 1:** Packet Loss Cause Analysis Process

**Input:** $(sip, dip)$: Source-destination IP pair extracted from NAK ;
**Output:** Loss analysis reports along the path;

**Localization Process**
    $switches \leftarrow$ `QueryFlowPath`$(sip, dip)$;
    $loss_reports \leftarrow \emptyset$;
    **foreach** $switch \in switches$ **do**
        $report \leftarrow$ `AnalyzeLoss`$(switch)$;
        $loss_reports \leftarrow loss_reports \cup \{report\}$;
    **return** $loss_reports$;

**Function** `AnalyzeLoss`$(dev)$
    **if** `LossAtASIC` *exists on dev* **then**
        **return** *Report(dev,* `LossAtASIC`*)*;
    **else if** `LossAtSwitch` *or* `LossFromProbe` *indicates loss on dev* **then**
        *anomaly* ← AnalyzeLogAndMetrics(`Logs` and `Perf. Metrics` on *dev*);
        **return** *Report(dev, anomaly)*;
    **else**
        **return** *None*;

**Algorithm 2:** Congestion Cause Analysis Process

**Input:** $(sip, dip)$: Source-destination IP pair reported as congested ;
*threshold*: FET threshold for incast detection ;
**Output:** Congestion analysis reports along the path;

**Localization Process**
    $links \leftarrow$ `QueryFlowPath`$(sip, dip)$;
    $cong_reports \leftarrow \emptyset$;
    **foreach** $link \in links$ **do**
        $report \leftarrow$ `AnalyzeCongestion`$(link)$;
        $cong_reports \leftarrow cong_reports \cup \{report\}$;
    **return** $cong_reports$;

**Function** `AnalyzeCongestion`$(link)$
    $flow_set \leftarrow$ QueryTraversingFlows($link$);
    **if** *at least 2 flows in* $flow_set$ *show FET* $\leq$ *threshold in the same time period* **then**
        **return** *Report(link,* $flow_set$*, "incast")*;
    **else if** $link.target$ *is NPU* ***and*** $link.source$ *keeps receiving PFC pause frames* **then**
        **return** *Report(link, NPU, "receiver-side throttling")*;
    **else**
        **return** *None*;

### 6.2 Device logs and performance metrics

Device logs are log entries collected from switches via Syslog [14]. Performance metrics are metrics reflecting switch health and behavior collected via SNMP [11] and similar telemetry methods. These metrics include, but are not limited to, interface statistics (e.g., interface state, discard counts), CPU/memory utilization, buffer usage, and BGP [21] session status. For congestion root-cause analysis, we additionally monitor four PFC-related metrics per RoCE switch queue at millisecond granularity, using a custom high-performance SNMP replacement: (1) PFC pause frames received, (2) PFC pause frames sent, (3) suspected PFC deadlock events, and (4) recoveries from suspected PFC deadlocks.

### 6.3 Loss root-cause analysis

NAK is the primary trigger for loss root-cause analysis because it covers all loss types. When an NAK is detected for a source-destination IP pair, the system first queries FlowPath to obtain the set of switches along the faulty path, then initiates the analysis process described in Algorithm 1.

The algorithm examines each switch along the path sequentially. If LossAtASIC reports are available for a switch, they are prioritized because they carry explicit root-cause information. Otherwise, the algorithm checks whether LossAtSwitch or LossFromProbe indicates packet loss on the device. If either does, a deeper analysis is conducted using device logs and performance metrics around the time of detection. We have accumulated a rule list that automatically match error patterns in logs and performance metrics: for example, the presence of `hwBoardFailure` in log indicates a hardware failure, while a sustained ingress error rate exceeding three packets per second for one minute on a switch port indicates a link CRC fault. The analysis results are included in the report. The algorithm description includes optional signals for completeness; when these signals are unavailable, the corresponding branches are simply skipped.

**Extreme cases.** For certain extreme faults such as silent corruption, NAK is the only fault signal available, and LossAtSwitch does not fire in these cases. In such cases, we intersect the set of paths that exhibit NAKs to narrow the fault scope to the most likely device, so that operators can isolate it as a temporary recovery method.

### 6.4 Congestion root-cause analysis

For each congested IP pair reported by Tier 1, Tier 2 first queries FlowPath to obtain the devices and links along the congested path, then draws on the recorded evidence (FlowPath, logs, performance metrics) over the corresponding time window to match against known congestion patterns.

The process is outlined in Algorithm 2. The algorithm examines each link along the path. For a given link, it queries FlowPath to identify all flows that traversed the link around

the time of congestion. If at least two of these flows exhibit FET below a predefined threshold (e.g., 70% of line rate) during that period, the algorithm infers incast congestion on that link and reports the link together with the affected flows. Otherwise, it checks whether the link's target device is an NPU and whether the upstream switch port is persistently receiving PFC pause frames during the congestion period. If both conditions hold, it infers receiver-side throttling and reports the link along with the NPU.

The root-cause analysis directly informs operator action. If all affected flows originate from the same tenant, the cause is likely an ECMP misconfiguration producing load imbalance. If the affected flows belong to multiple tenants, suboptimal affinity scheduling may be causing cross-tenant interference. Moreover, if performance metrics reveal BGP session anomalies or non-zero PFC deadlock counters, the underlying issue is BGP instability or a PFC deadlock, respectively.

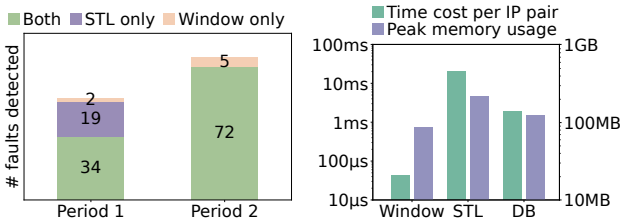
## 7 Deployment

FACT has been gradually deployed in multiple production AI cloud infrastructures. The scales of these infrastructure range from approximately 7,000 NPUs to more than 100,000 NPUs. The longest deployment has lasted 18 months. FACT covers all RoCE traffic in the deployed clusters.

**Processing mirrored packets at scale.** Handling large volumes of mirrored packets is challenging [44]. Theoretical estimation shows that a single ~7,000-NPU cluster can generate up to 10 Mpps of mirrored traffic under full load, corresponding to an estimated peak of 300 Gbps. We tested processing one-third of this peak on two CPU servers and observed CPU saturation leading to packet drops.

To reduce the processing burden on CPU servers, we pre-process mirrored packets on programmable switches. They truncate the packet payloads, timestamp the packets, and extract necessary fields at line rate before batch-uploading to CPU servers. In our deployment, two 800 Gbps programmable switches paired with four CPU servers can handle the peak mirrored traffic from four ~7,000-NPUs clusters. We emphasize that programmable switches are entirely optional: the same workload can be handled using CPU servers alone, at an estimated 90% higher cost. Deploying a larger number of programmable switches for preprocessing mirrored packets would split the mirrored traffic and shorten mirror paths, reducing the timestamp noise discussed in Lesson 3 (§9). However, the noise level under the current deployment is already acceptable.

**Avoiding impact on tenants.** Mirrored traffic that shares links with service flows is assigned lower scheduling priority to prevent impacting the service flows. Also, when LossFromProbe is available, probing intervals are on the order of seconds to minimize interference with service flows. Mirroring also raises no new privacy or security concerns because mirrored packets are payload-truncated.




(a) Accuracy (b) Overhead

**Figure 5. Accuracy and overhead of congestion detection algorithms. "Window" refers to sliding window, "STL" refers to STL decomposition, "DB" refers to dynamic baseline.**

**Loss–congestion interaction.** When FACT detects both loss and congestion on the same link simultaneously, it suppresses the congestion alert and retains only the loss alert. Our operational priority is to resolve the loss first: in our experience, resolving the loss typically also resolves the observed congestion.

## 8 Evaluation

First, we present micro-benchmarks of the congestion detection algorithms and the bandwidth overhead of packet mirroring (§8.1). Second, we present four case studies that illustrate end-to-end diagnosis (§8.2). Third, we report statistics from production deployment (§8.3).

### 8.1 Micro Benchmarks

**(Exp 1) Accuracy and overhead of congestion detection algorithms.** We evaluate the three congestion detection algorithms used in Tier 1: sliding window sampling, STL decomposition, and dynamic baseline. These algorithms were deployed at different times (sliding window and STL preceded dynamic baseline) so they were tested separately.

*Sliding window and STL.* We evaluate the two algorithms using real FET data from a 960-NPU tenant domain. Figure 5(a) compares their detection results over two periods. In a 12-hour period with sharp FET fluctuations, STL identified 53 faulty paths while sliding window detected 36; STL uniquely caught 19 cases obscured by fluctuations, while sliding window uniquely reported 2 short-duration anomalies. In a stable 36-hour period, sliding window reported 77 paths (5 more than STL's 72), capturing transient dips that STL overlooked. These results highlight the complementary strengths of both algorithms.

*Dynamic baseline.* We conducted fault injection experiments on a 32-NPU testbed, injecting four fault types affecting 12 source-destination IP pairs: slow links (2 pairs), switch port shutdowns (8 pairs), and 0.1% endpoint packet loss (2 pairs). The algorithm detected 11 of the 12 affected pairs (91.7%) with no false positives. The single missed detection occurred for an IP pair experiencing endpoint-side loss, where the FET fell slightly above the alert threshold.

*Computational and memory overhead.* We measure the average detection time per IP pair per cycle and peak memory

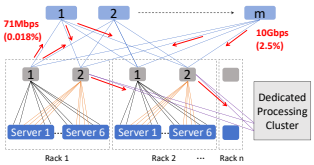



**Figure 6. Peak bandwidth utilization of mirroring RoCE packets.**

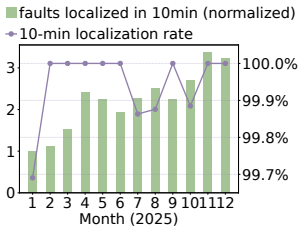



**Figure 7. Monthly count (normalized) of network faults localized within 10 minutes and the 10-minute fault localization rate in 2025.**

usage on an Intel i7 CPU, with approximately 2,000 IP pairs processed per cycle (Figure 5(b)). Sliding window sampling is the most lightweight: 44.3 us per pair and 87 MB peak memory. STL decomposition requires 20.3 ms per pair and 221 MB peak memory due to the complexity of the time-series decomposition. Dynamic baseline takes 1.93 ms per pair with 125 MB peak memory. All three exhibit acceptable overhead for large-scale deployment.

**(Exp 2) Bandwidth overhead of packet mirroring.** We evaluate the bandwidth consumed by mirroring selected RoCE packets (First, Last, and NAK) in a simulated environment emulating the traffic of a 125B-parameter large language model training run. We simulate the topology of Figure 1 with 12 spine switches and 144 racks (6,912 NPUs total). Two ToR switches (ToR 2 of rack 1 and rack 2) are connected to the cluster for processing mirrored packets, each using two 200Gbps links. We mirror selected RoCE packets on all ToR switches, and route them to the processing cluster through these two ToR switches, replicating the deployment in our production environment.

Figure 6 shows the peak bandwidth utilization. On the ToR-spine uplink, peak mirroring traffic reaches 71 Mbps (0.018% of link bandwidth). On the spine-ToR downlink, peak mirroring traffic reaches 10 Gbps (2.5% of link bandwidth). These results indicate that the bandwidth overhead of packet mirroring is acceptable. The total peak mirror traffic across the cluster reaches 240 Gbps, within the capacity of our processing infrastructure (§7).

### 8.2 Case Studies

We present four production cases that illustrate FACT's end-to-end diagnosis capability across diverse fault types.

**Case 1: Silent drops on a switch line card.** In a tenant domain with over 2,000 NPUs, NAKs were observed on three paths. LossAtSwitch triggered on one spine switch, while LossFromProbe cross-validated the result. Device logs and performance metrics provided no additional information, meaning that the drops were silent. We inspected the affected flow paths indicated by NAKs, and identified the specific line card on the affected spine switch that all three faulty paths traversed. The line card was isolated and replaced, resolving the fault. This case illustrates how LossAtSwitch combined with NAK-based path information enables fine-grained localization even for silent faults.

**Case 2: Silent corruption at low probability.** A training job experienced frequent exceptions without any LossAtSwitch or LossAtASIC alert. Frequent NAKs are monitored on a specific set of flows. By intersecting the paths of these flows, we identified a common spine switch as the likely faulty device. Neither LossAtSwitch nor LossFromProbe fired because the packets were not dropped, and no logs or metrics signaled an anomaly. We isolated the switch as a temporary recovery measure to restore service. Subsequent offline analysis revealed that packets passing through a specific memory buffer location were corrupted with a small probability, though the exact mechanism remains unclear. This case demonstrates NAK's unique coverage: it is the only signal that detects silent corruption, and combined with FlowPath-based tomography, it enables device-level localization even when no other signal is available.

**Case 3: ECMP polarization from tenant misconfiguration.** In a tenant domain with 360 NPUs, training throughput degraded persistently. All three congestion detection algorithms flagged multiple faulty paths, and PFC pause frames were observed on the affected links. Root-cause analysis identified a specific link carrying multiple flows with low FET, which points to an incast event. FlowPath revealed that all affected flows originated from the same tenant, but their paths deviated from the expected distribution. We communicated with the tenant and discovered a configuration error in their UDP source-port assignment that caused ECMP polarization. The tenant corrected the configuration, restoring load balance. This case shows the value of Tier 2's evidence chain: FlowPath identified which flows shared the congested link, FET confirmed the throughput degradation, and the combination pointed directly to a tenant-side root cause rather than an infrastructure fault.

**Case 4: Application slowdown from endpoint throttling.** A tenant with hundreds of NPUs reported persistent training linearity below 70%, without knowing which communication pairs were affected. Congestion detection identified the abnormal IP pairs. Root-cause analysis found no incast pattern, but the last-hop ToR switch persistently received PFC pause frames, indicating that the downstream NPU was failing to process packets at line rate. The issue was handed to the endpoint team for further investigation.

### 8.3 Long-term large-scale deployment

FACT has been deployed across multiple production clusters ranging from approximately 7,000 to over 100,000 NPUs.

**Detection and root-cause analysis accuracy.** We present detailed statistics from one representative ~12,000-NPU cluster over a 12-month period. Ground truth comes from our operations ticketing system: a fault enters the set only after

**Table 2.** Taxonomy of network faults we encountered in production. "Cong." refers to Congestion.

| Class | Root Cause |
|---|---|
| Loss | Physical link down. |
| | Physical link flapping. |
| | Misconfigured QoS directs RoCE traffic to lossy queue. |
| | Packet fails hardware check (e.g., checksum, parity). |
| | Switch buffer full. |
| | ACL rule denies forwarding. |
| | Missing forwarding table entry (e.g., MAC, ARP). |
| | Wrong or invalid forwarding table entry. |
| | Switch discards packets to suppress MAC flapping. |
| | Packet dropped due to port isolation. |
| | Silent drop (no explicit signal). |
| | Silent corruption (packet corrupted, not dropped). |
| Cong. | ECMP misconfiguration causing load imbalance. |
| | Suboptimal affinity scheduling causing cross-tenant interference. |
| | Receiver endpoint cannot process packets fast enough. |
| | BGP session failure or instability. |
| | PFC deadlock or PFC storm. |

its root cause was repaired and the affected tenant verified normal service (for congestion, training linearity returned to target). The recall denominator covers all tenant-visible incidents in the period. On this cluster, FACT achieved 100% detection recall across approximately 300 confirmed faults: every packet loss event was localized to the switch, and every congestion event originating from the network was traced to its root cause. Detection precision reached 99%; the 1% precision loss was traced to management-packet drops rather than service-traffic faults, which do not impact service workloads. Root-cause analysis pinpointed the cause for 99% of faults; the remaining 1% were silent faults localized to the device level but without specific root-cause labels.

**Congestion noise reduction.** In production, the congestion detection algorithms initially produced a 60% false-positive rate in production, caused by ERSPAN timestamp inaccuracy as described in §5.3. The severity filter (PFC pause frame cross-validation and tenant feedback) reduced the false-positive rate to below 10% at the Tier 1 output. Root-cause analysis in Tier 2 further reduced false positives to near zero, because false alerts carry no diagnosable root cause and are not labeled with any causes. For sustained congestion, the false-negative rate is near zero: periodic detection makes the probability of missing a congestion event decays rapidly across consecutive cycles.

**Faults detected.** Table 2 provides a taxonomy of the fault types detected by FACT in production. Link failures (physical link down, flapping, or CRC errors) are the most common cause of packet loss. For congestion, incast caused by ECMP imbalance or suboptimal scheduling accounts for over 95% of events. Receiver-side throttling accounts for most of the remaining share, with occasional QoS misconfigurations and BGP session anomalies appearing at low frequency.

**Rapid fault response.** Figure 7 shows the monthly count (normalized) of network faults localized within 10 minutes and the 10-minute fault localization rate across all deployed clusters in 2025. The fault count rose as FACT's deployment expanded. FACT achieves actionable localization for 99.95% of network faults within 10 minutes. (For congestion, the 10-minute clock starts when the severity filter in §5.3 reports the event, not when congestion first occurs.) Over 90% of faults are localized within one minute, though our records lack per-fault timing at percentile granularity. Packet loss events that generate abnormal logs or metrics, which are the majority, are typically localized to root causes within seconds. Silent faults are localized to the device within minutes, limited by the LossAtSwitch collection interval and manual NAK path analysis time. For these silent faults, operators can immediately isolate the faulty device and restore service, even if the exact internal failure mechanism remains under investigation. For congestion, once the severity filter reports a congestion event, root-cause analysis provides fine-grained localization within minutes, enabling operators to act promptly.

## 9 Lessons Learned

**Lesson 1: The best diagnostic signal is the one the system already generates.** NAK packets, packet conservation counters, and per-flow throughput are all obtainable from the network without new infrastructure, new protocols, or host instrumentation. They are already present in the network, waiting to be collected. We thus learned to ask *what does my system already produce that I can repurpose for diagnosis?* before asking *what new monitoring should I deploy?*

**Lesson 2: Distinguish detection from diagnosis.** Early in FACT's development, we attempted to drive the entire diagnostic pipeline from switch logs and performance metrics. But silent faults broke this approach completely. We once spent over 30 hours diagnosing a single silent corruption event. We therefore learned that logs and detection serve different purposes. Logs excel at root-cause labeling after detection and coarse localization, while detection requires objective and precise signals. Separating them into distinct tiers simplified the design, reduced false positives, and made the system's behavior easier to reason about. The separation assigns primary roles rather than forbidding overlap: high-confidence evidence can still support detection, as the PFC-based severity filter (§5.3) does.

**Lesson 3: The measurement pipeline limits accuracy more than the algorithm.** Our congestion detection algorithms performed well in testbed experiments but exhibited a non-negligible false-positive rate in production. The root cause was not algorithmic, but was that our ERSPAN [12] cannot timestamp on the switch, so network jitter on the mirror path distorted FCT and FET values. Our fix was conducting

a severity filter instead of designing a better algorithm. We thus learned that in measurement systems, the acquisition pipeline often matters more than the analysis algorithm.

**Lesson 4: Match diagnostic granularity to operational actionability.** For certain extreme faults such as silent corruption, NAK tells us that loss occurred on a specific path, but none of our other signals provide further detail. We initially invested effort in closing this gap, attempting to extract finer-grained information through more complex telemetry. We eventually realized that the operational action in these cases is always the same regardless of the exact internal failure mechanism: isolate the suspected component and replace it. Silent faults of this type are typically hardware faults originating in the data plane, so no amount of additional telemetry changes the operator's next step. Pursuing finer-grained diagnosis in real time added complexity without changing the operator's action. Diagnostic precision has value only when it enables a different response.

## 10 Generalizability

This section discusses how FACT's design can be extended beyond our current operational scenario, and where its assumptions impose fundamental limits.

**What FACT fundamentally requires.** The three signals rest on three conditions: NAK-based loss detection requires RoCE-RC transport and per-flow path affinity; LossAtSwitch requires packet conservation in the RoCE queue; FET requires long-lived, throughput-sensitive flows. Our deployment provides path affinity through pre-configured source ports and static ECMP, but any mechanism that keeps a flow on one reconstructable path would serve well.

**Generalizing to dynamic per-flow load balancing.** The design of FACT extends from static to dynamic per-flow ECMP. Under non-static per-flow schemes, the flow-to-path mapping changes over time, but we can still use packet sampling combined with ECMP hash calculation to record each flow's path. NAK remains a valid path-level loss indicator, LossAtSwitch is also unaffected, and FET computation remains valid. The only adjustment is in congestion detection: instead of aggregating FET values by source-destination IP pair, the detection algorithms must aggregate FET values by the path taken by each flow, since the path for a given IP pair may vary. All root-cause analysis evidence (FlowPath, logs, performance metrics) continues to apply.

**Generalizing to latency-bound workloads.** FACT currently targets throughput-bound AI training workloads by monitoring FET. However, the design principle of obtaining diagnostic data using only commodity-switch primitives enables us to also collect latency information for congestion detection in latency-sensitive workloads. For example, following the approach of EverFlow [44], we can sample a subset of packets, tag them with a debug marker at the first-hop switch, and mirror each tagged packet at every subsequent hop along its path together with a per-hop timestamp. This yields per-hop queuing delay, which directly reveals congestion locations for latency-bound traffic. We leave this as future work to combine latency and throughput signals to provide unified fault detection across a broader range of workload types.

**Mixed fabrics.** Our RoCE fabric is a dedicated scale-out backend, so every loss in the RoCE queue signals a fault. A mixed fabric preserves this premise only with isolated lossless queues and buffers; otherwise loss cause analysis must also rule out shared-buffer pressure and QoS scheduling.

**Limitations.** Two scenarios lie beyond FACT's reach. First, packet spraying—where packets of a single flow are distributed across multiple paths—renders NAK unreliable as a loss indicator because reordering can trigger NAKs without packet loss, and per-path FET analysis becomes infeasible since flow throughput is no longer tied to a single path. Second, FACT's detection mechanisms are designed for RoCE semantics (NAK-based loss signaling, PFC-driven losslessness, and RC-mode flow boundaries). They do not directly apply to TCP or other transport protocols without substantial redesign. Within the scope of per-flow RoCE networks, FACT's design generalizes to a broad class of topologies, load-balancing schemes, and workload profiles.

## 11 Conclusion

This paper presented FACT, a network fault detection and localization system for RoCE-based AI cloud infrastructures that operates under public-cloud and commodity-hardware constraints. It utilizes a two-tier architecture that separates detection with root-cause analysis, and exploits fault signals that are protocol-native and can detect throughput degradation. FACT has been deployed in production for 18 months multiple production infrastructures. In 2025, it achieved actionable localization for 99.95% of network faults within 10 minutes. Beyond the specific mechanisms, we believe the operational lessons distilled from this deployment generalize to other large-scale network diagnosis efforts.

## Acknowledgments

We thank our anonymous ATC shepherd and the anonymous reviewers for their valuable feedback. This work was supported by the National Key Research and Development Program of China (2023YFB2904600), the Joint Funds of the National Natural Science Foundation of China (U25B2038), and the Research Grants Council of Hong Kong (GRF 14201523).

## References

[1] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. 2008. A scalable, commodity data center network architecture. In *Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication (SIGCOMM '08)*. Association for Computing Machinery, New York, NY, USA, 63–74. https://doi.org/10.1145/1402958.1402967

[2] Behnaz Arzani, Selim Ciraci, Luiz Chamon, Yibo Zhu,

Hongqiang (Harry) Liu, Jitu Padhye, Boon Thau Loo, and Geoff Outhred. 2018. 007: Democratically Finding the Cause of Packet Drops. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, Renton, WA, 419–435. https://www.usenix.org/conference/nsdi18/presentation/arzani

[3] Chuhao Chen, Jiarui Ye, Yongbo Gao, Sen Liu, and Yang Xu. 2024. HFˆ2T: Host-Based Flowlet Fine-Tuning for RDMA Load Balancing. In *Proceedings of the 8th Asia-Pacific Workshop on Networking (APNet '24)*. Association for Computing Machinery, New York, NY, USA, 9–15. https://doi.org/10.1145/3663408.3663410

[4] Benoît Claise. 2004. Cisco Systems NetFlow Services Export Version 9. RFC 3954. (Oct. 2004). https://doi.org/10.17487/RFC3954

[5] David D. Clark, Craig Partridge, J. Christopher Ramming, and John T. Wroclawski. 2003. A knowledge plane for the internet. In *Proceedings of the 2003 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications (SIGCOMM '03)*. Association for Computing Machinery, New York, NY, USA, 3–10. https://doi.org/10.1145/863955.863957

[6] Robert B Cleveland, William S Cleveland, Jean E McRae, Irma Terpenning, et al. 1990. STL: A seasonal-trend decomposition. *J. off. Stat* 6, 1 (1990), 3–73.

[7] Charles Clos. 1953. A study of non-blocking switching networks. *The Bell System Technical Journal* 32, 2 (1953), 406–424. https://doi.org/10.1002/j.1538-7305.1953.tb01433.x

[8] Rui Ding, Xunpeng Liu, Shibo Yang, Qun Huang, Baoshu Xie, Ronghua Sun, Zhi Zhang, and Bolong Cui. 2024. RD-Probe: Scalable Monitoring With Sufficient Coverage In Complex Datacenter Networks. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 258–273. https://doi.org/10.1145/3651890.3672256

[9] Jianbo Dong, Kun Qian, Pengcheng Zhang, Zhilong Zheng, Liang Chen, Fei Feng, Yichi Xu, Yikai Zhu, Gang Lu, Xue Li, Zhihui Ren, Zhicheng Wang, Bin Luo, Peng Zhang, Yang Liu, Yanqing Chen, Yu Guan, Weicheng Wang, Chaojie Yang, Yang Zhang, Man Yuan, Hanyu Zhao, Yong Li, Zihan Zhao, Shan Li, Xianlong Zeng, Zhiping Yao, Binzhang Fu, Ennan Zhai, Wei Lin, Chao Wang, and Dennis Cai. 2025. Evolution of Aegis: Fault Diagnosis for AI Model Training Service in Production. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, Philadelphia, PA, 865–881. https://www.usenix.org/conference/nsdi25/presentation/dong

[10] D. Farinacci, T. Li, S. Hanks, D. Meyer, and P. Traina. 2000. RFC2784: Generic Routing Encapsulation (GRE). (2000).

[11] Mark Fedor, Martin Lee Schoffstall, James R. Davin, and Dr. Jeff D. Case. 1990. Simple Network Management Protocol (SNMP). RFC 1157. (May 1990). https://doi.org/10.17487/RFC1157

[12] Marco Foschiano, Kalyan Ghosh, and Munish Mehta. 2017. *Cisco Systems' Encapsulated Remote Switch Port Analyzer (ERSPAN)*. Internet-Draft draft-foschiano-erspan-03. Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-foschiano-erspan/03/ Work in Progress.

[13] Yilong Geng, Shiyu Liu, Zi Yin, Ashish Naik, Balaji Prabhakar, Mendel Rosenblum, and Amin Vahdat. 2019. SIMON: a simple and scalable method for sensing, inference and measurement in data center networks. In *Proceedings of the 16th USENIX Conference on Networked Systems Design and Implementation (NSDI'19)*. USENIX Association, USA, 549–564.

[14] Rainer Gerhards. 2009. The Syslog Protocol. RFC 5424. (March 2009). https://doi.org/10.17487/RFC5424

[15] Yu Guan, Zhiyu Yin, Haoyu Chen, Sheng Cheng, Chaojie Yang, Kun Qian, Tianyin Xu, Pengcheng Zhang, Yang Zhang, Hanyu Zhao, Yong Li, Dennis Cai, and Ennan Zhai. 2026. EROICA: Online Performance Troubleshooting for Large-scale Model Training. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, Renton, WA, 1113–1130. https://www.usenix.org/conference/nsdi26/presentation/guan-yu

[16] Chuanxiong Guo, Lihua Yuan, Dong Xiang, Yingnong Dang, Ray Huang, Dave Maltz, Zhaoyi Liu, Vin Wang, Bin Pang, Hua Chen, Zhi-Wei Lin, and Varugis Kurien. 2015. Pingmesh: A Large-Scale System for Data Center Network Latency Measurement and Analysis. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication (SIGCOMM '15)*. Association for Computing Machinery, New York, NY, USA, 139–152. https://doi.org/10.1145/2785956.2787496

[17] Shixian Guo, Kefei Liu, Yulin Lai, Yangyang Bai, Ziwei Zhao, Songlin Liu, Jianghang Ning, Gen Li, Jianwei Hu, Yongbin Dong, Feng Luo, Sisi Wen, Qi Zhang, Yuan Chen, Jiale Feng, Yang Bai, Chengcai Yao, Zhe Liu, Xin Hu, Yang Lv, Zhuo Jiang, Jiao Zhang, and Tao Huang. 2025. ByteTracker: An Agentless and Real-time Path-aware Network Probing System. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 541–553. https://doi.org/10.1145/3718958.3750515

[18] Christian Hopps. 2000. Analysis of an Equal-Cost Multi-Path Algorithm. RFC 2992. (Nov. 2000). https://doi.org/10.17487/RFC2992

[19] Christian Hopps and Dave Thaler. 2000. Multipath Issues in Unicast and Multicast Next-Hop Selection. RFC 2991. (Nov. 2000). https://doi.org/10.17487/RFC2991

[20] IEEE. 2011. IEEE 802.1Qbb-2011 - Priority-Based Flow Control. (2011).

[21] Petr Lapukhov, Ariff Premji, and Jon Mitchell. 2016. Use of BGP for Routing in Large-Scale Data Centers. RFC 7938. (Aug. 2016). https://doi.org/10.17487/RFC7938

[22] Yuliang Li, Rui Miao, Changhoon Kim, and Minlan Yu. 2016. FlowRadar: A Better NetFlow for Data Centers. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*. USENIX Association, Santa Clara, CA, 311–324. https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/li-yuliang

[23] Yuliang Li, Rui Miao, Changhoon Kim, and Minlan Yu. 2016. LossRadar: Fast Detection of Lost Packets in Data Center Networks. In *Proceedings of the 12th International on Conference on Emerging Networking EXperiments and Technologies (CoNEXT '16)*. Association for Computing Machinery, New York, NY, USA, 481–495. https://doi.org/10.1145/2999572.2999609

[24] Kefei Liu, Zhuo Jiang, Jiao Zhang, Shixian Guo, Xuan Zhang, Yangyang Bai, Yongbin Dong, Feng Luo, Zhang Zhang, Lei Wang, Xiang Shi, Haohan Xu, Yang Bai, Dongyang Song, Haoran Wei, Bo Li, Yongchen Pan, Tian Pan, and Tao Huang. 2024. R-Pingmesh: A Service-Aware RoCE Network Monitoring and Diagnostic System. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, New York, NY, USA, 554–567. https://doi.org/10.1145/3651890.3672264

[25] Wei Liu, Kun Qian, Zhenhua Li, Tianyin Xu, Yunhao Liu, Weicheng Wang, Yun Zhang, Jiakang Li, Shuhong Zhu, Xue Li, Hongfei Xu, Fei Feng, and Ennan Zhai. 2025. SkeletonHunter: Diagnosing and Localizing Network Failures in Containerized Large Model Training. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 527–540. https://doi.org/10.1145/3718958.3750513

[26] Meta. 2016. NetNORAD: Troubleshooting Networks via End-to-end Probing. https://engineering.fb.com/2016/02/18/core-infra/netnorad-troubleshooting-networks-via-end-to-end-probing/. (2016).

[27] Radhika Mittal, Vinh The Lam, Nandita Dukkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. 2015. TIMELY: RTT-based Congestion Control for the Datacenter. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication (SIGCOMM '15)*. Association for Computing Machinery, New York, NY, USA, 537–550. https://doi.org/10.1145/2785956.2787510

[28] Masoud Moshref, Minlan Yu, Ramesh Govindan, and Amin Vahdat. 2016. Trumpet: Timely and Precise Triggers in Data Centers. In *Proceedings of the 2016 ACM SIGCOMM Conference (SIGCOMM '16)*. As-

sociation for Computing Machinery, New York, NY, USA, 129–143. https://doi.org/10.1145/2934872.2934879

[29] Sonia Panchen, Neil McKee, and Peter Phaal. 2001. InMon Corporation's sFlow: A Method for Monitoring Traffic in Switched and Routed Networks. RFC 3176. (Sept. 2001). https://doi.org/10.17487/RFC3176

[30] Yanghua Peng, Ji Yang, Chuan Wu, Chuanxiong Guo, Chengchen Hu, and Zongpeng Li. 2017. deTector: a Topology-aware Monitoring System for Data Center Networks. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. USENIX Association, Santa Clara, CA, 55–68. https://www.usenix.org/conference/atc17/technical-sessions/presentation/peng

[31] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, and Alex C. Snoeren. 2017. Passive Realtime Datacenter Fault Detection and Localization. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. USENIX Association, Boston, MA, 595–612. https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/roy

[32] Praveen Tammana, Rachit Agarwal, and Myungjin Lee. 2016. Simplifying Datacenter Network Debugging with PathDump. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, Savannah, GA, 233–248. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/tammana

[33] Cheng Tan, Ze Jin, Chuanxiong Guo, Tianrong Zhang, Haitao Wu, Karl Deng, Dongming Bi, and Dong Xiang. 2019. NetBouncer: Active Device and Link Failure Localization in Data Center Networks. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. USENIX Association, Boston, MA, 599–614. https://www.usenix.org/conference/nsdi19/presentation/tan

[34] Erico Vanini, Rong Pan, Mohammad Alizadeh, Parvin Taheri, and Tom Edsall. 2017. Let it flow: resilient asymmetric load balancing with flowlet switching. In *Proceedings of the 14th USENIX Conference on Networked Systems Design and Implementation (NSDI'17)*. USENIX Association, USA, 407–420.

[35] Chenxu Wang, Xumiao Zhang, Runwei Lu, Xianshang Lin, Xuan Zeng, Xinlei Zhang, Zhe An, Gongwei Wu, Jiaqi Gao, Chen Tian, Guihai Chen, Guyue Liu, Yuhong Liao, Tao Lin, Dennis Cai, and Ennan Zhai. 2025. Towards LLM-Based Failure Localization in Production-Scale Networks. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 496–511. https://doi.org/10.1145/3718958.3750505

[36] Shicheng Wang, Menghao Zhang, Xiao Li, Qiyang Peng, Haoyuan Yu, Zhiliang Wang, Mingwei Xu, Xiaohe Hu, Jiahai Yang, and Xingang Shi. 2025. Hawkeye: Diagnosing RDMA Network Performance Anomalies with PFC Provenance. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 481–495. https://doi.org/10.1145/3718958.3750490

[37] Weitao Wang, Xinyu Crystal Wu, Praveen Tammana, Ang Chen, and T. S. Eugene Ng. 2022. Closed-loop Network Performance Monitoring and Diagnosis with SpiderMon. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 267–285. https://www.usenix.org/conference/nsdi22/presentation/wang-weitao-spidermon

[38] Bo Yang, Huanwu Hu, Yifan Li, Yunguang Li, Xiangyu Tang, Bingchuan Tian, Gongwei Wu, Jianfeng Xu, Xumiao Zhang, Feng Chen, Cheng Wang, Ennan Zhai, Yuhong Liao, Dennis Cai, and Tao Lin. 2025. SkyNet: Analyzing Alert Flooding from Severe Network Failures in Large Cloud Infrastructures. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 512–526. https://doi.org/10.1145/3718958.3750536

[39] Zhiyi Yao, Pengbo Hu, Congcong Miao, Xuya Jia, Zuning Liang, Yuedong Xu, Chunzhi He, Hao Lu, Mingzhuo Chen, Xiang Li, Zekun He, Yachen Wang, Xianneng Zou, and Junchen Jiang. 2025. Holmes: Localizing Irregularities in LLM Training with Mega-scale GPU Clusters. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, Philadelphia, PA, 523–540. https://www.usenix.org/conference/nsdi25/presentation/yao

[40] Yiran Zhang, Yifan Liu, Qingkai Meng, and Fengyuan Ren. 2021. Congestion detection in lossless networks. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference (SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 370–383. https://doi.org/10.1145/3452296.3472899

[41] Yu Zhou, Chen Sun, Hongqiang Harry Liu, Rui Miao, Shi Bai, Bo Li, Zhilong Zheng, Lingjun Zhu, Zhen Shen, Yongqing Xi, Pengcheng Zhang, Dennis Cai, Ming Zhang, and Mingwei Xu. 2020. Flow Event Telemetry on Programmable Data Plane. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '20)*. Association for Computing Machinery, New York, NY, USA, 76–89. https://doi.org/10.1145/3387514.3406214

[42] Shunmin Zhu, Jianyuan Lu, Biao Lyu, Tian Pan, Chenhao Jia, Xin Cheng, Daxiang Kang, Yilong Lv, Fukun Yang, Xiaobo Xue, Zhiliang Wang, and Jiahai Yang. 2022. Zoonet: a proactive telemetry system for large-scale cloud networks. In *Proceedings of the 18th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT '22)*. Association for Computing Machinery, New York, NY, USA, 321–336. https://doi.org/10.1145/3555050.3569116

[43] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. 2015. Congestion Control for Large-Scale RDMA Deployments. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication (SIGCOMM '15)*. Association for Computing Machinery, New York, NY, USA, 523–536. https://doi.org/10.1145/2785956.2787484

[44] Yibo Zhu, Nanxi Kang, Jiaxin Cao, Albert Greenberg, Guohan Lu, Ratul Mahajan, Dave Maltz, Lihua Yuan, Ming Zhang, Ben Y. Zhao, and Haitao Zheng. 2015. Packet-Level Telemetry in Large Datacenter Networks. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication (SIGCOMM '15)*. Association for Computing Machinery, New York, NY, USA, 479–491. https://doi.org/10.1145/2785956.2787483

# A Appendix

## A.1 LossFromProbe

LossFromProbe refers to indicators of packet loss generated by end-to-end active probing. It helps to localize packet loss to specific devices. Due to the end-to-end nature of probing, it can cover all types of packet loss. It serves as a cross-validation mechanism for LossAtSwitch and NAK. We developed a pingmesh-like [16] solution to monitor it when deploying probing agents are authorized by tenants.

**Approach.** We deploy lightweight probing agents on tenant hosts to cover the complete paths traversed by service RoCE flows. Upon tenant domain creation, a pinglist is generated to cover all possible paths traversed by inter-ToR and intra-ToR service flows within the tenant domain. Agents periodically execute the probe tasks in the pinglist and generate alerts when the packet loss rate of a probe task exceeds configured thresholds. An analyzer collects these alerts, analyzes their paths, and applies a voting algorithm to identify the most likely faulty devices. These devices are then marked as having LossFromProbe.

**Architectural overview.** Our solution operates through three primary components: the *controller*, the *agents* and the *analyzer*. The controller generates a detailed *pinglist* that

specifies the probe tasks required to ensure comprehensive coverage. Agents are deployed on hosts. They periodically execute the probe tasks and generate alerts. The analyzer collects the alerts from agents and analyzes them to generate LossFromProbe.

**Controller.** The controller generates pinglists each time when computational resources are allocated to a tenant. Each pinglist entry specifies a sender-to-receiver probe task. As depicted in Figure 1, our cluster architecture features 2-tier Clos topology with each rack containing 6 servers and 2 ToR switches. Within each server, NPUs with ranks 0,1,6,7 connect exclusively to ToR 1 while ranks 2,3,4,5 connect to ToR 2. This creates two natural *NPU groups* per server, each comprising four NPUs sharing the same ToR. Since our minimum resource allocation unit is a physical server, all NPUs within a server are allocated to the same tenant. The controller constructs two complementary probe sets:

- **Intra-ToR pinglist:** This implements ring-based probing within NPU groups (e.g., 0→1→6→7→0 sequence). By enforcing ToR traversal, these probes ensure coverage of all NPUs, RNICs and server-ToR downlinks. Each intra-group cycle generates exactly one probe task per NPU within the tenant domain.
- **Inter-ToR pinglist:** This establishes cross-group probing between NPU groups following rank-specific patterns (e.g., a rank 0 NPU targets all other rank 0/2 NPUs). Tasks where the source and destination NPUs connect to the same ToR are excluded. This approach validates spine switch functionality and spine-ToR uplinks in the tenant service network.

**Agent.** Agents are deployed within tenant containers to perform periodic network probing. They require tenant authorization to utilize NPU resources for probing. Through a dedicated NPU probing interface we developed, these agents generate customized RoCE probe packets directly from hardware. Each probe packet is configured with specific UDP source port field to ensure it follows the same ECMP path as tenant traffic originating from the same NPU. Agents continuously monitor packet loss rates of each probe task. They generate alerts when the loss rate of a probe task exceeds configured thresholds (e.g., >50% packet loss in one minute).

**Analyzer.** The analyzer periodically collects alerts from agents and analyzes them. The analysis process involves several key steps:

1. **Path determining**: For alerted probe tasks, the analyzer first determines their network paths. We periodically conduct full-mesh `traceroute` several times a day within each tenant service network to record up-to-date path information for all possible source-destination IP pairs. The analyzer utilizes these `traceroute` results to determine the paths of alerted probe tasks with high accuracy.
2. **Representative alert selection**: To eliminate redundant alerts and reduce processing volume, the analyzer selects a minimal set of representative probe tasks from all alerts. This selection process counts device occurrence frequency across alert paths, sorts devices in descending order, and chooses one alert per device using conditional rules prioritizing high-probability fault coverage.
3. **Loss indicator generation**: Using paths of representative alerts, the analyzer applies a voting algorithm to identify the most frequently occurring devices in these paths as primary suspects. The number of identified suspicious devices is configurable, typically set less than three. The analyzer then reports LossFromProbe for these devices.